

**DESIGN ENGINEERING PROJECT**

Course Code: CP301

# Smart Irrigation System

IoT-Based Automated Field Moisture Management

---

Submitted by

**Rahul**

Roll No: 2023EEB1238

**Kunal Dhiman**

Roll No: 2023EEB1218

---

Academic Year 2025–2026

Department of Electronics Engineering | Computer Engineering

## Project Information

|                  |                                                 |
|------------------|-------------------------------------------------|
| Project Title    | Smart Irrigation System                         |
| Course           | Development Engineering Project (DEP)           |
| Course Code      | CP301                                           |
| Academic Year    | 2025–2026                                       |
| Student 1        | Rahul   Roll No: 2023EEB1238                    |
| Student 2        | Kunal Dhiman   Roll No: 2023EEB1218             |
| Department       | Electronics & Computer Engineering              |
| Technology Stack | ESP32, AWS IoT Core, MQTT Protocol, Mobile App  |
| Communication    | MQTT over Wi-Fi (Cloud-based, replaced ESP-NOW) |
| Threshold        | 73% Soil Moisture                               |

## Abstract

Agriculture accounts for nearly 70% of global freshwater consumption, and a significant portion is wasted due to over-irrigation or improper scheduling. This project presents a fully functional IoT-based Smart Irrigation System designed to address this challenge by automating water delivery based on real-time soil moisture data.

The system employs two ESP32 microcontrollers communicating over a cloud-based MQTT infrastructure hosted on Amazon Web Services (AWS IoT Core). One ESP32 unit, deployed in the field, continuously reads soil moisture levels. When moisture falls below the pre-configured threshold of 73%, the device publishes an alert to the cloud, triggering the second ESP32 to actuate a solenoid valve and activate the irrigation motor. A companion mobile application provides real-time visibility into soil conditions and motor/valve status from anywhere.

This project demonstrates a complete hardware-software co-design solution that eliminates range limitations of peer-to-peer communication (ESP-NOW was evaluated and replaced), while laying a scalable groundwork for future enhancements including solar-powered operation and AI-driven predictive scheduling.

# 1. Introduction

---

## 1.1 Background and Motivation

Water scarcity is one of the most pressing environmental challenges of the 21st century. In India, agriculture is the backbone of the economy, and yet traditional flood irrigation methods lose up to 50% of water to evaporation, runoff, and deep percolation. Farmers often rely on fixed schedules or intuition rather than actual soil conditions, leading to crop damage, water wastage, and higher operational costs.

Smart irrigation, enabled by the Internet of Things (IoT), offers a transformative solution. By embedding sensors directly into the field and connecting them to intelligent control systems via the cloud, irrigation can be triggered precisely when and where it is needed.

## 1.2 Problem Statement

Conventional irrigation systems suffer from three key problems:

- Lack of real-time data: Farmers have no reliable way to monitor field moisture remotely.
- Manual operation: Irrigation motors must be switched on and off manually, requiring constant presence in the field.
- Communication range limitations: Low-cost peer-to-peer wireless protocols like ESP-NOW are limited to a few hundred metres, making them unsuitable for larger field deployments.

## 1.3 Project Objectives

- Design and build a dual-node IoT system capable of sensing and actuating across distances without range restrictions.
- Implement cloud-based communication using MQTT and AWS IoT Core for reliable, scalable messaging.
- Develop a real-time mobile application to monitor soil moisture and irrigation status.
- Build a fully working physical prototype with actual sensor and motor hardware.
- Lay the foundation for future enhancements such as solar power integration and predictive analytics.

## 2. System Architecture

---

The system is built around two geographically separated ESP32 nodes that communicate indirectly through the AWS IoT Core cloud broker. This architecture decouples the sensor node from the actuator node, allowing them to operate independently and at any distance from each other as long as Wi-Fi connectivity is available.

### 2.1 High-Level Architecture

The overall architecture consists of three tiers:

- Field Node (Sensor ESP32): Monitors soil moisture continuously and publishes data to the cloud.
- Cloud Broker (AWS IoT Core / MQTT): Acts as a secure, scalable intermediary that routes messages between nodes.
- Control Node (Actuator ESP32): Subscribes to cloud messages and controls the irrigation motor/solenoid valve.
- Mobile Application: Subscribes to the same MQTT topics to display real-time moisture readings and device status.

### 2.2 Data Flow

The data flow for a typical irrigation cycle is as follows:

- Step 1: The capacitive soil moisture sensor on Node A continuously reads the volumetric water content of the soil.
- Step 2: The ESP32 on Node A publishes moisture percentage values to a designated MQTT topic on the AWS IoT Core broker at regular intervals (e.g., every 30 seconds).
- Step 3: When moisture drops below 73%, Node A publishes a trigger message to a separate MQTT control topic.
- Step 4: Node B, subscribed to the control topic, receives the trigger and activates the relay, opening the solenoid valve and starting the motor.
- Step 5: Node B publishes its status (motor ON / OFF, valve state) back to a status topic.
- Step 6: The mobile app reads both moisture data and status topics, displaying live readings to the user.
- Step 7: Once the moisture level rises above the threshold, Node A publishes a deactivation message, and Node B closes the valve.

### 2.3 Why Cloud MQTT Over ESP-NOW

Initially, the team prototyped the system using ESP-NOW, Espressif's proprietary peer-to-peer protocol. While ESP-NOW delivered very low latency communication, it was found to have a practical line-of-sight range of 200–400 metres, which is insufficient for real field deployments where the pump house and the field may be kilometres apart. By migrating to MQTT over AWS IoT Core, the system achieves:

- Unlimited range: Communication works anywhere both nodes have internet connectivity.
- Message persistence: AWS IoT Core retains messages, preventing data loss during brief network interruptions.
- Security: All communication is encrypted using TLS/SSL with X.509 certificates.
- Scalability: Additional nodes (multiple fields, multiple sensors) can be added without re-designing the architecture.

### 3. Hardware Components

The following hardware components were selected for their reliability, community support, and suitability for embedded IoT applications.

| Component                           | Specification / Role                           | Quantity    |
|-------------------------------------|------------------------------------------------|-------------|
| ESP32 Development Board             | Dual-core MCU with built-in Wi-Fi & Bluetooth  | 2           |
| Capacitive Soil Moisture Sensor     | Measures volumetric soil moisture (0–100%)     | 1           |
| 5V Single-Channel Relay Module      | Controls motor/solenoid valve on/off switching | 1           |
| Solenoid Valve (12V DC)             | Electronically controlled water flow valve     | 1           |
| DC Water Pump / Motor               | Field irrigation pump simulated in prototype   | 1           |
| 3.7V Li-Po Battery + TP4056 Charger | Portable power for field sensor node           | 1           |
| 5V USB Power Supply                 | Powers the control node and relay              | 1           |
| Jumper Wires & Breadboard           | Prototyping connections                        | As required |
| PVC Pipes & Fittings                | Physical water delivery in the prototype       | As required |
| Android/iOS Smartphone              | Runs the companion mobile application          | 1           |

#### 3.1 ESP32 Microcontroller

The ESP32 (by Espressif Systems) was chosen as the primary microcontroller for both nodes due to its dual-core 240 MHz Xtensa LX6 processor, integrated 802.11 b/g/n Wi-Fi, Bluetooth 4.2/BLE, and rich peripheral support including ADC, GPIO, I2C, SPI, and UART. Its low cost and extensive library ecosystem make it ideal for IoT prototyping.

#### 3.2 Capacitive Soil Moisture Sensor

Unlike resistive sensors, which corrode over time due to electrolysis, the capacitive soil moisture sensor measures the dielectric permittivity of the surrounding soil to determine its water content. The sensor outputs an analogue voltage proportional to moisture level, which the ESP32 reads through its built-in 12-bit ADC. The output is calibrated to produce a percentage value (0% = completely dry, 100% = fully saturated). The irrigation threshold is set at 73%.

#### 3.3 Solenoid Valve and Relay

A normally-closed 12V DC solenoid valve controls the water supply line. Since the ESP32 GPIO pins cannot directly drive the solenoid coil, a 5V relay module is interposed. When the control ESP32 asserts a GPIO pin HIGH (or LOW depending on relay polarity), the relay coil energises, closing the circuit to the solenoid valve and allowing water to flow.

## 4. Software Design

---

### 4.1 Firmware – Sensor Node (Node A)

The firmware for the sensor ESP32 is written using the Arduino framework in C/C++. On boot, the device connects to a configured Wi-Fi network and establishes a secure MQTT connection with the AWS IoT Core endpoint using device-specific X.509 certificates stored in flash memory. It then enters a measurement loop that:

- Reads the raw ADC value from the soil moisture sensor every 30 seconds.
- Converts the ADC value to a percentage using a two-point linear calibration (dry reference = 0%, wet reference = 100%).
- Publishes the JSON-encoded moisture reading to the topic: irrigation/sensor/moisture.
- Evaluates whether the moisture level is below 73%. If true, publishes a motor ON command to: irrigation/control/motor.
- If moisture recovers above 73%, publishes a motor OFF command to the same topic.

### 4.2 Firmware – Actuator Node (Node B)

Node B subscribes to the irrigation/control/motor MQTT topic on startup. Upon receiving a motor ON payload, it asserts the relay GPIO pin to energise the solenoid valve and activate the pump. It publishes its status back to irrigation/status/motor so the mobile app can reflect the current state. On receiving a motor OFF payload, it de-energises the relay.

### 4.3 AWS IoT Core Configuration

AWS IoT Core was configured with the following resources:

- Thing Registry: Two IoT Things registered (SensorNode and ActuatorNode), each with unique certificates and private keys.
- IoT Policy: Attached policies grant each Thing permission to publish and subscribe only to its relevant topics.
- MQTT Broker Endpoint: Both devices connect to the region-specific AWS endpoint over port 8883 (MQTTS).
- Topic Rules (optional): AWS IoT Rules Engine can be configured to log data to DynamoDB or trigger SNS alerts for historical analysis.

### 4.4 Mobile Application

A companion mobile application was developed to provide end-users with a real-time dashboard. The app connects to the same MQTT broker and subscribes to:



- irrigation/sensor/moisture — to display live soil moisture percentage with a gauge or progress bar.
- irrigation/status/motor — to display the current motor and valve state (ON/OFF indicator).

Additional features implemented in the app include:

- Historical trend chart showing moisture variation over the last 24 hours.
- Manual override controls allowing the user to force motor ON or OFF from the app.
- Push notification alert when moisture drops below threshold or rises above safe level.
- Wi-Fi signal strength indicator for both nodes (via a status heartbeat topic).

## 5. Working Prototype

---

A fully functional physical prototype was built and demonstrated. The prototype replicates a real-world deployment on a reduced scale:

### 5.1 Physical Setup

- A small container filled with soil acts as the test field. The capacitive moisture sensor is inserted vertically into the soil to a depth of 5 cm.
- The sensor ESP32 (Node A) is powered by a USB power bank, simulating field deployment with a battery.
- Node B (Actuator ESP32) is placed at a different location and connected to a 5V relay, which in turn drives a small DC water pump immersed in a water reservoir.
- PVC tubing channels water from the reservoir to the soil container, completing the irrigation loop.
- Both nodes are connected to separate Wi-Fi networks (or the same router in the demo), demonstrating that communication is purely cloud-mediated.

### 5.2 Demonstration Results

During testing and demonstration, the following results were recorded:

- Response latency from moisture drop detection to motor activation: approximately 1.5–3 seconds (dependent on network conditions).
- Moisture threshold accuracy: The system consistently activated irrigation at 73% and deactivated once moisture recovered.
- App update latency: Mobile app received moisture and status updates within 2 seconds of ESP32 publishing.
- System stability: Prototype ran continuously for several hours without requiring a reset.
- Manual override: App-initiated manual ON/OFF commands were executed reliably.

## 6. Protocol Comparison: ESP-NOW vs. MQTT over AWS

| Parameter              | ESP-NOW                   | MQTT over AWS IoT            |
|------------------------|---------------------------|------------------------------|
| Range                  | 200–400 m (line of sight) | Unlimited (internet-based)   |
| Infrastructure Needed  | None (peer-to-peer)       | Wi-Fi + AWS account          |
| Latency                | < 10 ms                   | 1–3 seconds                  |
| Security               | Basic (MAC-based)         | TLS/SSL + X.509 certificates |
| Scalability            | Limited (max 20 peers)    | Highly scalable (cloud)      |
| Message Persistence    | None                      | Retained messages supported  |
| Cost                   | Free                      | AWS Free Tier (low cost)     |
| Mobile App Integration | Not directly possible     | Native MQTT support          |

## 7. Challenges Faced

The development process involved significant problem-solving across multiple domains:

- **ESP-NOW Range Limitation:** Initial prototyping with ESP-NOW revealed communication failure beyond ~300 metres. The team had to redesign the communication layer, migrating to MQTT over AWS IoT Core. This involved learning AWS console configuration, certificate management, and secure MQTT connection setup from scratch.
- **ADC Calibration:** The ESP32 ADC has known non-linearities. Raw readings required two-point calibration against verified dry and saturated soil samples to achieve accurate percentage readings.
- **Wi-Fi Reconnection Stability:** Early firmware suffered from connection drops. The team implemented exponential back-off reconnection logic and watchdog timer resets to ensure nodes recovered automatically.
- **Relay and Solenoid Noise:** Electromagnetic interference from the relay coil caused sporadic ESP32 resets. This was resolved by adding a flyback diode across the relay coil and adding a 100  $\mu$ F capacitor on the 5V supply line.
- **Mobile App MQTT Integration:** Establishing authenticated MQTT connections in a mobile environment (with certificate pinning) required careful handling of TLS context and broker hostname resolution.

- Power Management for Field Node: Keeping Node A battery-powered for extended periods required implementing deep-sleep mode between readings, significantly reducing idle current consumption.

## 8. Future Enhancements

---

The current system provides a robust baseline. The following upgrades are planned:

### 8.1 Solar Power Integration

The field sensor node (Node A) will be upgraded with a solar charging circuit comprising a 5W solar panel, a CN3791 MPPT solar charge controller, and a 3.7V 3000 mAh Li-Po battery. This will make the sensor node completely self-sufficient, eliminating the need for manual recharging and enabling permanent field deployment.

### 8.2 Multiple Sensor Nodes

A single moisture reading may not represent the entire field uniformly. The architecture will be extended to support multiple sensor nodes at different points in the field, with the cloud aggregating readings and triggering irrigation only in zones where moisture is below threshold.

### 8.3 AI/ML-Based Predictive Irrigation

By storing historical moisture and weather data in AWS DynamoDB, a machine learning model (e.g., LSTM or Random Forest) can predict future soil moisture trends based on temperature, humidity, and past patterns. This would allow pre-emptive irrigation before soil becomes critically dry, further optimising water usage.

### 8.4 Weather API Integration

The system will integrate with a public weather API (e.g., OpenWeatherMap). If rainfall is forecast within the next 6–12 hours, the system will suppress irrigation, avoiding unnecessary water usage.

### 8.5 LoRa / NB-IoT Support

For fields without Wi-Fi coverage, a LoRa (Long Range) radio module can be added to Node A, using a LoRaWAN gateway to relay data to the internet. This extends deployment viability to remote agricultural areas.

### 8.6 Dashboard and Analytics Web Portal

An AWS-hosted web dashboard (using AWS Amplify and AppSync) will allow farm managers to monitor multiple fields, review historical data, set per-zone thresholds, and generate water consumption reports.

### 8.7 Fertigation Integration

Future iterations will incorporate a secondary valve controlling liquid fertiliser injection into the irrigation line. The system can then deliver both water and nutrients on demand, maximising crop yield.

## 9. Conclusion

---

This project successfully demonstrates a complete, end-to-end IoT-based smart irrigation system that automates water delivery based on real-time soil moisture sensing. The transition from ESP-NOW to MQTT over AWS IoT Core was a critical design decision that elevated the system from a short-range prototype to a genuinely deployable agricultural tool.

The working physical prototype, combined with a functional mobile application, validates the entire data pipeline from field sensor to cloud broker to actuator to user interface. The system achieved consistent performance: irrigation is triggered reliably at the 73% moisture threshold, and the mobile app reflects live status within seconds of any change.

Beyond its technical achievements, this project offers a tangible solution to a real-world problem — water waste in agriculture. With the planned enhancements including solar power, multiple zones, predictive analytics, and weather integration, the system has the potential to be scaled into a commercially viable precision irrigation platform.

This project provided invaluable hands-on experience in IoT hardware design, embedded firmware development, cloud infrastructure configuration, and mobile application development, reinforcing concepts taught across multiple courses in the curriculum.

## 10. References

---

- Espressif Systems. ESP32 Technical Reference Manual. <https://www.espressif.com/en/support/documents/technical-documents>
- Amazon Web Services. AWS IoT Core Developer Guide. <https://docs.aws.amazon.com/iot/latest/developerguide/>
- OASIS Open. MQTT Version 3.1.1 Specification. <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/>
- Espressif. ESP-NOW User Guide. [https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp\\_now.html](https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html)
- Arduino. Arduino Framework for ESP32. <https://github.com/espressif/arduino-esp32>
- PubSubClient Library. MQTT Client for Arduino. <https://pubsubclient.knolleary.net/>
- Figma. UI/UX Design for Mobile Application. <https://www.figma.com>

- National Water Mission, Government of India. Efficient Irrigation and Water Conservation. <https://nwm.gov.in>